

# Lab Report 5: MOSFET Characterization

**Name:** Connor Savugot

**Due Date:** March 18, 2025

**Course:** ECE 302

## Introduction

In this laboratory, you will measure the current-voltage characteristics of MOSFETs in several regions of operation. Some of the large signal parameters will be determined from the measured data. It is highly recommended you record results in a spreadsheet to help with the necessary calculations. All resistors should be measured beforehand using the multimeter to obtain precise current calculations.

The HEF4007 chip that will be used in this laboratory contains three nMOSFETs and three pMOSFETs. A schematic of the chip is shown below. One-third of the total devices have the source tied to the bulk. Whether the MOSFET is n- or p-channel is indicated by an arrow on either the source or on the bulk. The HEF4007 chip has no “power” or “ground” pins – the MOSFETs are biased individually.

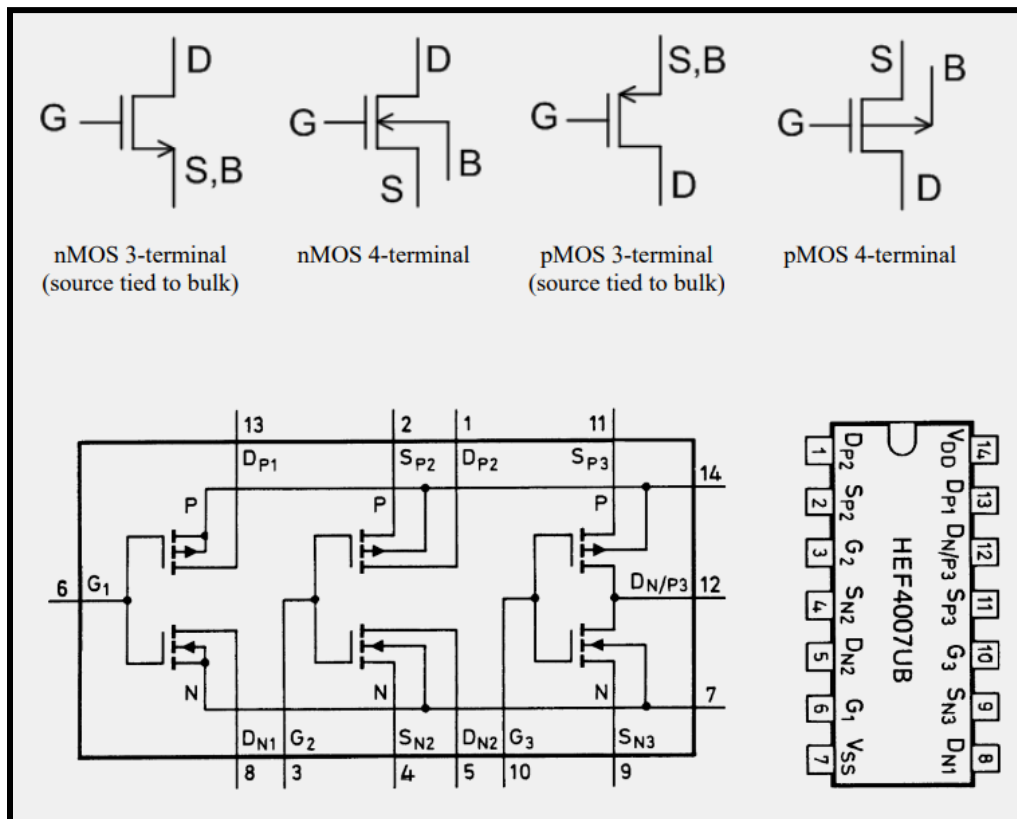


Figure 1.1: MOSFET IC Diagram

## In-Lab Work and Data

### 3.0 Family of Curves (*nMOS and pMOS*)

A MOSFET's "family of curves" refers to a plot of drain current versus drain-to-source voltage for multiple values of  $V_{GS}$ . In sections 3.1 and 3.2, the goal is to record enough data points to create a plot of  $I_{DS}$  versus  $V_{DS}$  at four different  $V_{GS}$  values. For future analysis, it would be convenient to save your data in a spreadsheet.

#### 3.1 Experiment A (*nMOS*)

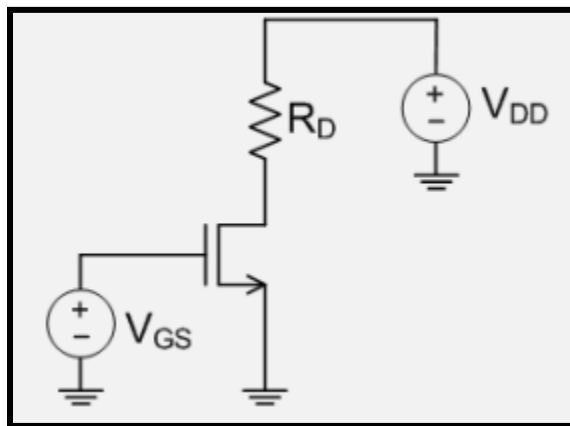


Figure 3.1 Circuit Schematic

1. Build circuit 3.1 with  $R_D=500\Omega$  (two 1k resistors in parallel).
2. Connect the V+ wire to the top terminal of  $R_D$  and ground wire to pin 7(source) on the HEF4007 IC.
3. Connect the bottom terminal of  $R_D$  with pin 8(drain) and connect W2 wire to pin 6(gate).
4. Open Wavegen in Waveforms and select W2 to produce a constant voltage (DC) of 3V to  $V_{GS}$ .
5. Connect 1+ and 1- wires across drain and source terminals of the MOSFET to measure the  $V_{DS}$  value.
6. Increase the  $V_{DD}$  (supplied by V+) from 0V to obtain  $V_{DS}=0.5V$ . Record  $V_{DD}$  and  $V_{DS}$  values. Use KVL to calculate the voltage drop across  $R_D$  and then use Ohm's law to find the current flowing through  $R_D$ . Repeat this process until  $V_{DS}=5V$ , incrementing by 0.5V.
7. Repeat step 7 for  $V_{GS}=4V$  and 5V.

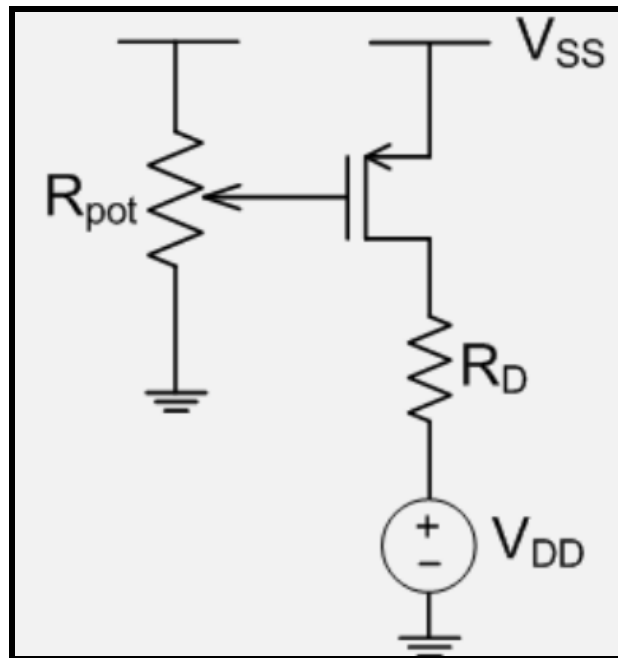
Note: You may supply VDD using the W1 source in AD2. The advantage of using V+ is that the slider present in Waveforms makes it easier to tune the VDD values while observing the VDS values on the voltmeter. (1+ and 1-). However, V+ has a minimum value of 0.5V. W1 allows you to set values between 0 and 0.5V. You may find that V+ reaches its maximum value of 5V but the VDS is not yet close to 5V. In such a case, you need to increase the voltage beyond 5V by utilizing V- . Notes: You may supply VDD using the W1 source in AD2. The advantage of using V+ is that the slider present in Waveforms makes it easier to tune the VDD values while observing the VDS values on the voltmeter. (1+ and 1-). However, V+ has a minimum value of 0.5V. W1 allows you to set values between 0 and 0.5V. You may find that V+ reaches its maximum value of 5V but the VDS is not yet close to 5V. In such a case, you need to increase the voltage beyond 5V by utilizing V- .

In case of  $V_{GS} = 4V$  &  $V_{GS} = 5V$ , you will need to increase your VDD beyond 5V to observe VDS values greater than  $\sim 3V$ . To build a VDD that can supply voltages greater than 5V, you can combine W1 and V+ channels.

It is important to realize that in doing so, you will change your ground connection.

- Remove the AD2 ground connection from the circuit.
- Connect W1 to the ground on the circuit (in this case, the source terminal) and set it to
- 4V in case of  $V_{GS}=4V$  and -5V in case of  $V_{GS}=5V$ .
- Set W2 (connected to pin6) to 0V. Although W2 is connected to gate, it is set 0V with respect to AD2 ground. Since the circuit ground is now W1 which is at
- $V_{GS}$  with respect to AD2 ground, effective voltage at gate is  $V_{GS}$  with respect to circuit ground. - Start the V+ slider from 500mV and repeat step 7. Note that total VDD will be  $V+ - W1$ . For a W1 of -4V and a V+ of 3V, the VDD becomes 7V

### **3.2 Experiment B (pMOS)**



*Figure 3.2 Circuit Schematic*

Using the circuit in Figure 3.2, sweep  $V_{DS}$  from 0 to -5V by -0.5V for  $V_{GS} = -3, -4, -5V$ . To achieve every  $V_{GS}$  value, you will need to rotate  $R_{pot}$ . Use  $V_{SS} = 5V$ ,  $R_D = 500\Omega$ , and any size potentiometer.

Again,  $V_{DD}$  is the voltage source swept to vary  $V_{DS}$ . Adjust  $V_{DS}$  in small steps until the drain current saturates, then use larger steps. Update a plot as you work so that a suitable step size can be chosen.

1. Connect the circuit according to Figure 3.2. Use W1 for  $V_{SS}$  to supply 5V to the potentiometer as well as to the source (pin 14). Connect the middle pin of the potentiometer to the gate (pin 6) and the other pins of the potentiometer to  $V_{SS}$  and ground.
2. Connect  $R_D$  to the drain (pin 13) of the PMOSFET.
3. Use V+ as  $V_{DD}$  to connect to  $R_D$ .
4. Use 1+ and 1- wires to measure  $V_{GS}$  value and adjust the potentiometer to get  $V_{GS} = -3V$ .
5. Use 2+ and 2- wires to measure  $V_{SD}$  value.
6. Start  $V_{DD}$  at 5V and adjust it to get a  $V_{SD} = 0V$ .
7. Start decreasing  $V_{DD}$  to get  $V_{SD} = 0.5V$ . Note down  $V_{DD}$  and  $V_{SD}$  and use KVL to calculate  $I_D$  value. Repeat this process in 0.5V increments until  $V_{SD} = 5V$ . You might have to make  $V_{DD}$  slightly negative to get  $V_{SD} = 5V$ .

8. Repeat step 7 for  $V_{GS} = -4\text{ V}$  and  $-5\text{ V}$ .

Note: Most likely, you will need to enter negative values of  $V_{DD}$  in order to get  $V_{SD}$  to  $5\text{ V}$ . Since negative voltages as well as voltages between  $-0.5\text{ V}$  &  $+0.5\text{ V}$  may be needed, you could setup  $V_{DD}$  in one of the following 2 ways:

A.  $V_+$  for  $0.5\text{ V} < V_{DD} < 5\text{ V}$

W2 for  $-0.5\text{ V} < V_{DD} < 0.5\text{ V}$

$V_-$  for  $-5\text{ V} < V_{DD} < -0.5\text{ V}$

B. Connect W2 for the entire range. This means you won't need to switch between supplies but you will also not have the convenience of the slider.

## 4.0 Measurement with DCA Pro

Follow the DCA Pro measurement guide using the same MOSFETs as used in sections 3.1 and 3.2, as individual chips can vary. You should end up with a data for  $I_{DS}$  vs.  $V_{GS}$ , and a data for  $I_D$  vs  $V_{DS}$  for the nMOS device and a similar dataset for the pMOS device. Setup the measurement as follows for both the nMOS and the pMOS:

| Plot type        | $V_{GS}$        | $V_{DS}$      |
|------------------|-----------------|---------------|
| $I_D$ - $V_{GS}$ | 0-5V; 51 points | 0.1V; 1 trace |

| Plot type        | $V_{GS}$       | $V_{DD}$      |
|------------------|----------------|---------------|
| $I_D$ - $V_{DS}$ | 0-5V; 6 traces | 0-12V; 51 pts |

The raw data will be in a text file that can be loaded into MS Excel or MATLAB. To answer the post lab questions, see the supplemental information document "Lab 5 MOSFET Parameter Extraction".

## Data Sets and Graphs

### 3.1 Experiment A (nMOS)

| $V_{GS} = 3\text{ V}$ |                           |                            |                 | $V_{GS} = 4\text{ V}$ |                           |                            |                 | $V_{GS} = 5\text{ V}$ |                           |                            |                 |
|-----------------------|---------------------------|----------------------------|-----------------|-----------------------|---------------------------|----------------------------|-----------------|-----------------------|---------------------------|----------------------------|-----------------|
| $V_{DD}(\text{V})$    | Actual $V_{DS}(\text{V})$ | Desired $V_{DS}(\text{V})$ | $I_D(\text{A})$ | $V_{DD}(\text{V})$    | Actual $V_{DS}(\text{V})$ | Desired $V_{DS}(\text{V})$ | $I_D(\text{A})$ | $V_{DD}(\text{V})$    | Actual $V_{DS}(\text{V})$ | Desired $V_{DS}(\text{V})$ | $I_D(\text{A})$ |
| 1.115                 | 0.502                     | 0.5                        | 0.001226        | 1.6                   | 0.506                     | 0.5                        | 0.002188        | 2.025                 | 0.5                       | 0.5                        | 0.00305         |
| 1.75                  | 0.998                     | 1                          | 0.001504        | 2.75                  | 0.998                     | 1                          | 0.003504        | 3.7                   | 0.996                     | 1                          | 0.005408        |
| 2.28                  | 1.504                     | 1.5                        | 0.001552        | 3.465                 | 1.5                       | 1.5                        | 0.00393         | 4.88                  | 1.504                     | 1.5                        | 0.006752        |

|       |       |     |          |       |       |     |              |       |       |     |              |
|-------|-------|-----|----------|-------|-------|-----|--------------|-------|-------|-----|--------------|
| 2.8   | 2.008 | 2   | 0.001584 | 4.02  | 1.998 | 2   | 0.00404<br>4 | 5.63  | 2.006 | 2   | 0.00724<br>8 |
| 3.3   | 2.5   | 2.5 | 0.0016   | 4.56  | 2.5   | 2.5 | 0.00412      | 6.22  | 2.508 | 2.5 | 0.00742<br>4 |
| 3.821 | 3.012 | 3   | 0.001618 | 5.08  | 2.998 | 3   | 0.00416<br>4 | 6.755 | 2.996 | 3   | 0.00751<br>8 |
| 4.32  | 3.506 | 3.5 | 0.001628 | 5.6   | 3.498 | 3.5 | 0.00420<br>4 | 7.3   | 3.504 | 3.5 | 0.00759<br>2 |
| 4.82  | 3.998 | 4   | 0.001644 | 6.13  | 4.008 | 4   | 0.00424<br>4 | 7.825 | 4     | 4   | 0.00765      |
| 5.325 | 4.508 | 4.5 | 0.001634 | 6.63  | 4.492 | 4.5 | 0.00427<br>6 | 8.35  | 4.498 | 4.5 | 0.00770<br>4 |
| 5.83  | 5.004 | 5   | 0.001652 | 7.152 | 4.998 | 5   | 0.00430<br>8 | 8.876 | 5     | 5   | 0.00775<br>2 |

*Table 4.1 nMOS Data*

### 3.2 Experiment B (pMOS)

| VGS=-3V |                  |          | VGS=-4V |                  |          | VGS=-5V |                  |          |
|---------|------------------|----------|---------|------------------|----------|---------|------------------|----------|
| VDD(V)  | Actual<br>VSD(V) | Id (A)   | VDD(V)  | Actual<br>VSD(V) | Id       | VDD(V)  | Actual<br>VSD(V) | Id       |
| 4.2     | 0.506            | 0.000588 | 4       | 0.492            | 0.001016 | 3.8     | 0.502            | 0.001396 |
| 3.55    | 0.996            | 0.000908 | 3.05    | 1.014            | 0.001872 | 2.7     | 0.99             | 0.00262  |
| 2.98    | 1.506            | 0.001028 | 2.3     | 1.502            | 0.002396 | 1.7     | 1.498            | 0.003604 |
| 2.46    | 1.998            | 0.001084 | 1.65    | 2.006            | 0.002688 | 0.85    | 1.994            | 0.004312 |
| 1.94    | 2.498            | 0.001124 | 1.09    | 2.494            | 0.002832 | 0.1     | 2.508            | 0.004784 |
| 1.43    | 2.99             | 0.00116  | 0.55    | 2.986            | 0.002928 | -0.52   | 2.996            | 0.005048 |
| 0.9     | 3.506            | 0.001188 | 0       | 3.494            | 0.003012 | -1.12   | 3.506            | 0.005228 |
| 0.385   | 4.004            | 0.001222 | -0.55   | 4.008            | 0.003084 | -1.7    | 4.014            | 0.005372 |
| -0.125  | 4.5              | 0.00125  | -1.1    | 4.522            | 0.003156 | -2.25   | 4.508            | 0.005484 |
| -0.65   | 5.014            | 0.001272 | -1.6    | 4.996            | 0.003208 | -2.8    | 5.008            | 0.005584 |

*Table 4.2 nMOS Data*

### 4.0 Measurement with DCA Pro

**nMOS Data:**

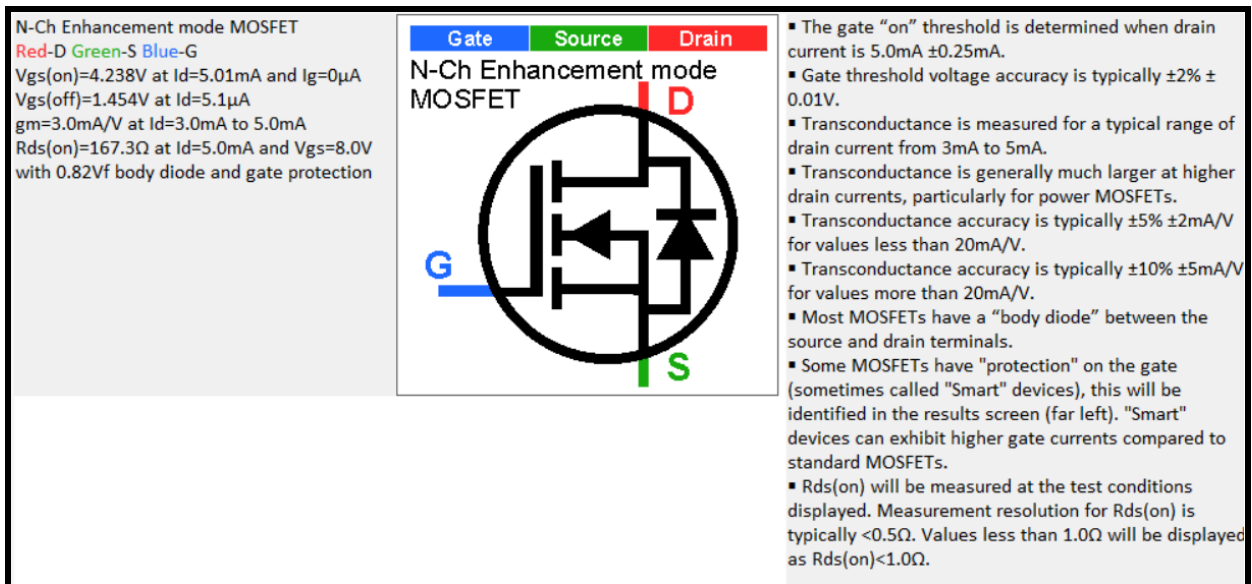


Figure 4.3 nMOS DCA Pro Overview

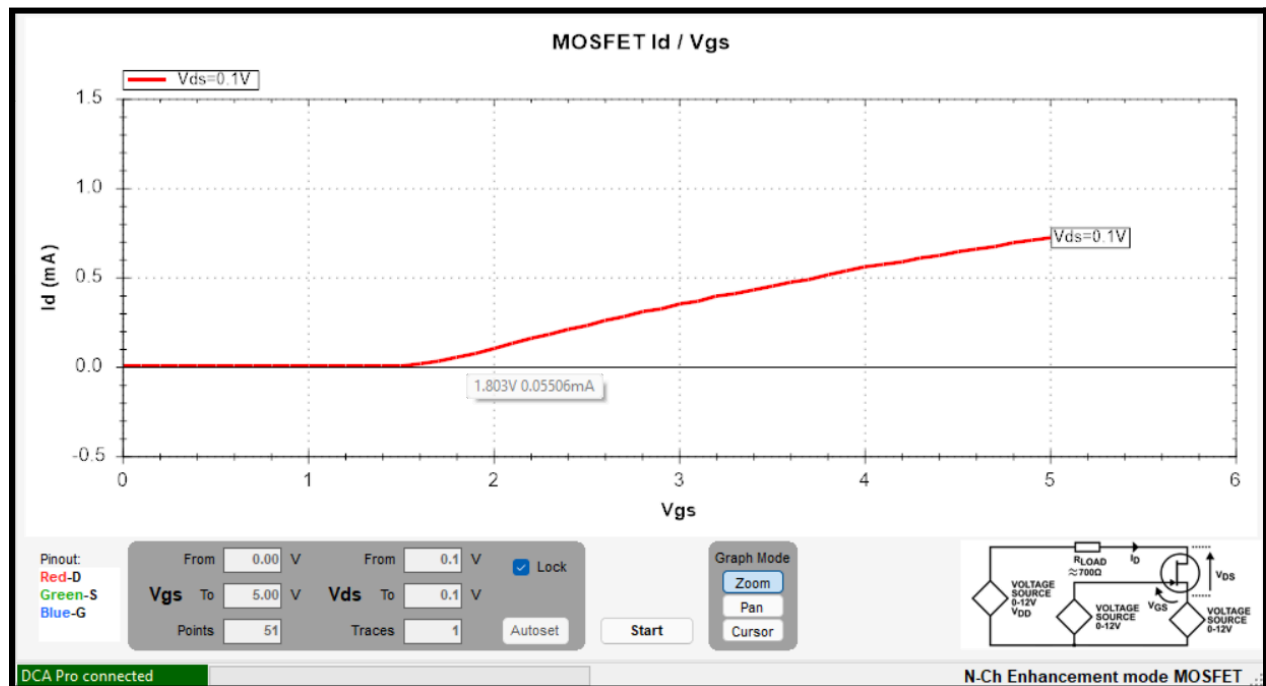


Figure 4.4 nMOS DCA Pro  $I_d$  vs  $V_{gs}$

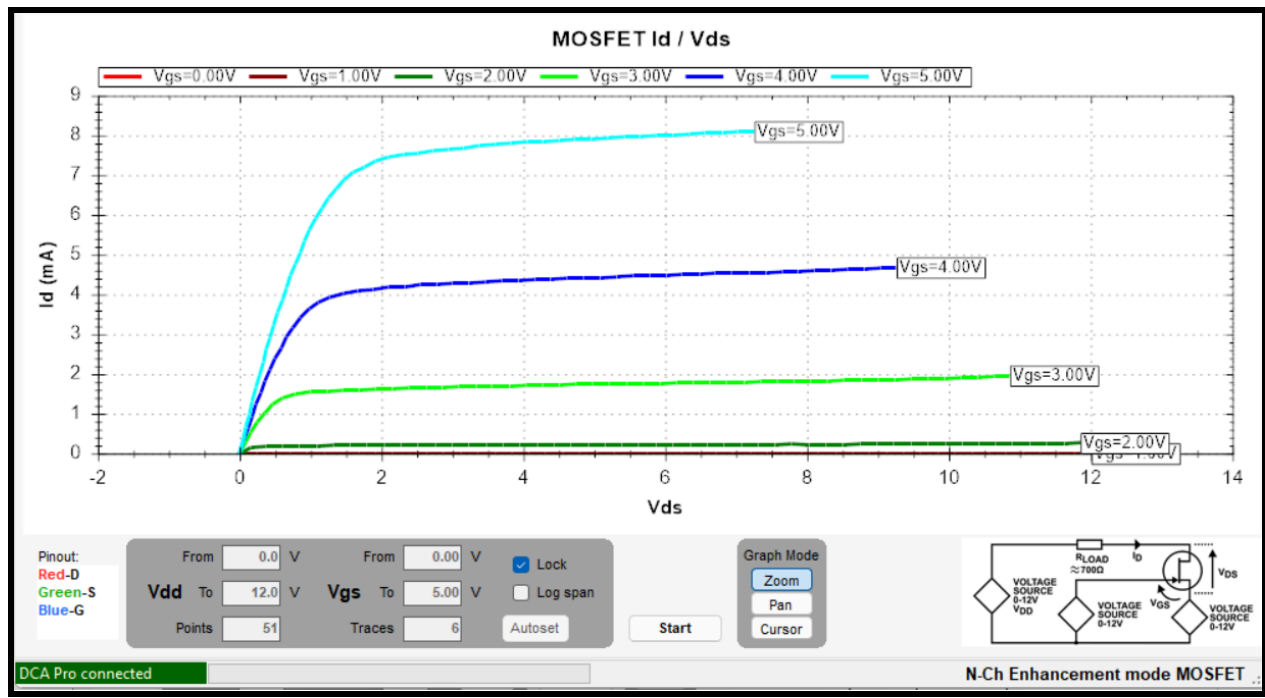


Figure 4.5 nMOS DCA Pro Id vs Vds

## pMOS Data:

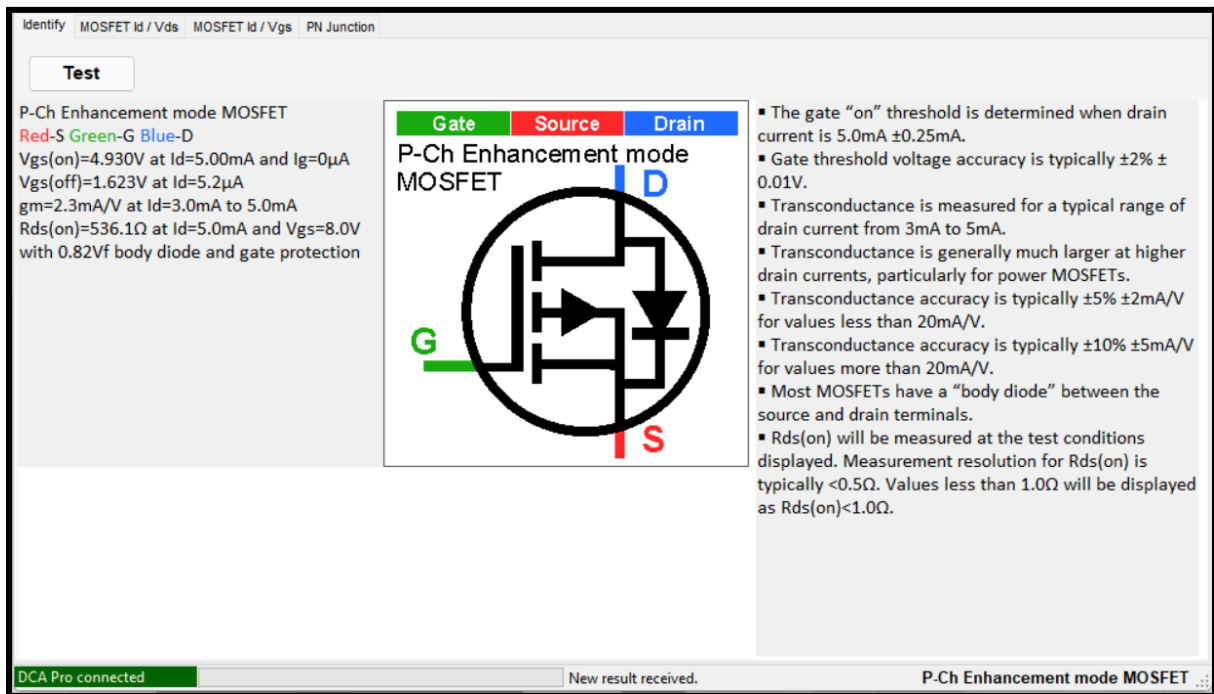


Figure 4.6 pMOS DCA Pro Overview

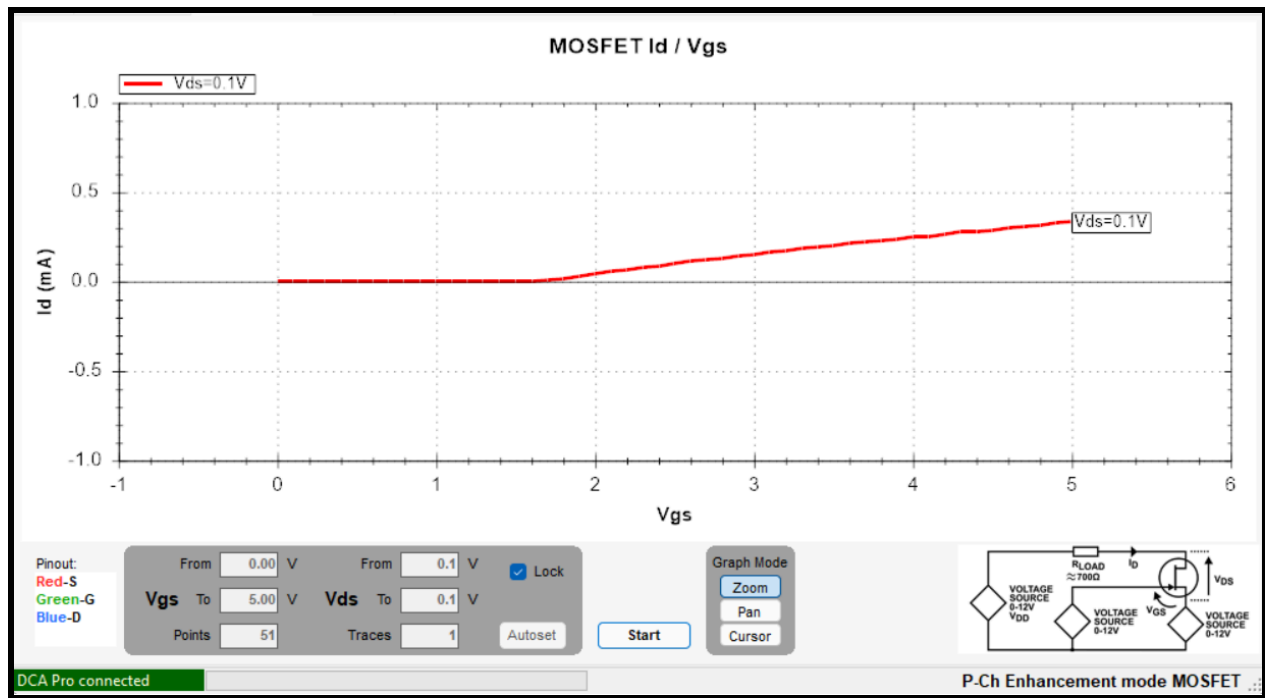


Figure 4.7 pMOS DCA Pro Id vs Vgs

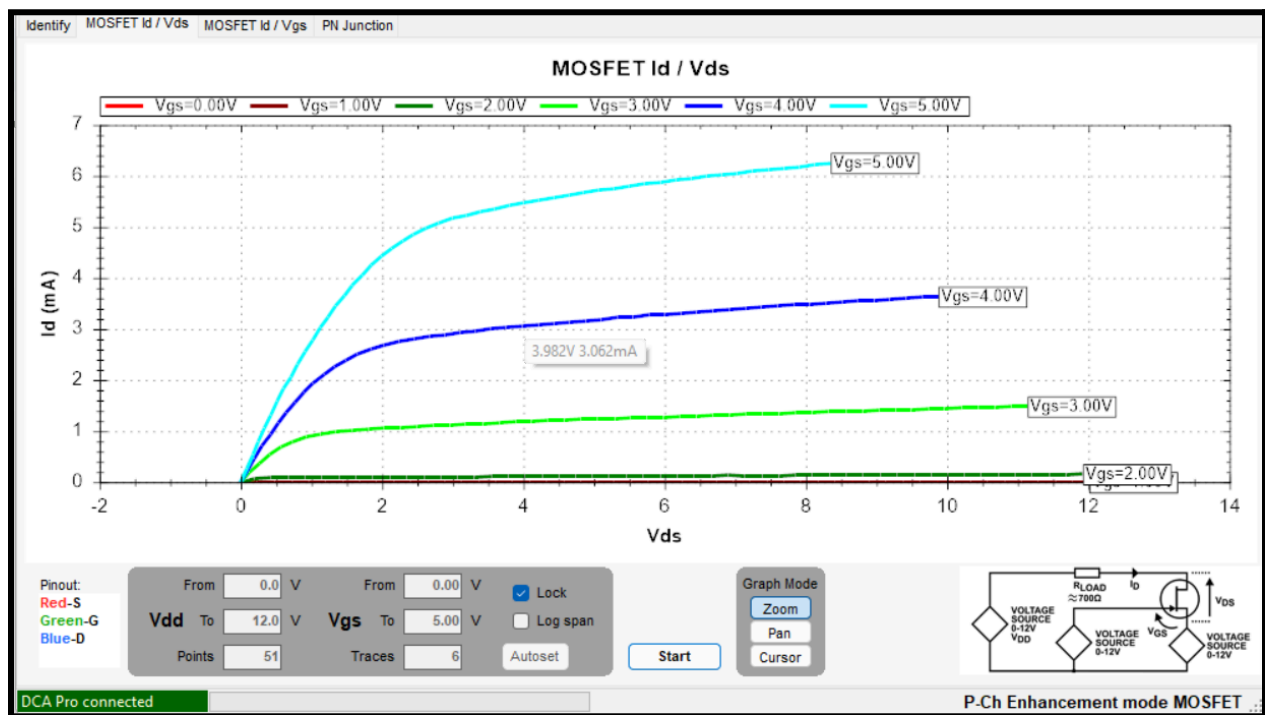


Figure 4.8 pMOS DCA Pro Id vs Vds

## Post-Lab and Analysis

### Section 3.0

Question 1: Extract an average value for  $\lambda$  from  $I_{DS}$  versus  $V_{DS}$  at  $V_{GS} = 3, 4, 5$  V (nMOS).

Extract an average value for  $\lambda$  from  $I_{DS}$  versus  $V_{DS}$  at  $V_{GS} = -3, -4, -5$  V (pMOS).

Average Lambda Values:

nMOS:  $\lambda_{avg} = 1.388e-02$

pMOS:  $\lambda_{avg} = 5.367e-02$

### Code:

```
%% Lab5_Postlab1
% Reset workspace
clear all; close all; clc;
%% Load Data from CSV Files
% Read nMOS data (skipping the first row which contains headers)
nMOS_data = readmatrix('nMOS.csv');
% Extract columns based on nMOS structure
V_GS_n = nMOS_data(2:end, 1); % Gate-Source Voltage (V_GS)
V_DD_n = nMOS_data(2:end, 2); % Supply Voltage (V_DD)
V_DS_n = nMOS_data(2:end, 3); % Drain-Source Voltage (V_DS)
I_DS_n = nMOS_data(2:end, 4); % Drain Current (I_DS)
% Read pMOS data (skipping the first row which contains headers)
pMOS_data = readmatrix('pMOS.csv');
% Extract columns based on pMOS structure
V_GS_p = pMOS_data(2:end, 1); % Gate-Source Voltage (V_GS)
V_DD_p = pMOS_data(2:end, 2); % Supply Voltage (V_DD)
V_SD_p = pMOS_data(2:end, 3); % Source-Drain Voltage (V_SD)
I_SD_p = pMOS_data(2:end, 4); % Source Current (I_SD)
% Convert pMOS parameters to match nMOS format
V_DS_p = -V_SD_p; % Convert V_SD to V_DS
I_DS_p = I_SD_p; % No need to negate current; it's already in the correct
direction
%% Extract Lambda (Channel Length Modulation) for nMOS
VGS_values_n = [3, 4, 5]; % nMOS target V_GS values
lambda_n_values = [];
for VGS_target = VGS_values_n
    idx_vgs = abs(V_GS_n - VGS_target) < 0.1; % Select data for this V_GS
    idx_sat = idx_vgs & (V_DS_n > 3); % Further filter to saturation region

    % Debugging output
    fprintf('\nExtracting lambda for nMOS at V_GS = %d V:\n', VGS_target);
    disp([V_DS_n(idx_sat), I_DS_n(idx_sat)]); % Print selected data
    if sum(idx_sat) >= 2
        fit_lambda = polyfit(V_DS_n(idx_sat), I_DS_n(idx_sat), 1);
        lambda_n_values(end+1) = fit_lambda(1) / fit_lambda(2);
    else
```

```

        disp('Warning: Not enough data to extract lambda for this V_GS.');
```

end

```

end

lambda_n_avg = mean(lambda_n_values, 'omitnan'); % Compute average, ignoring
NaNs

%% Extract Lambda (Channel Length Modulation) for pMOS
VGS_values_p = [-3, -4, -5]; % pMOS target V_GS values
lambda_p_values = [];
for VGS_target = VGS_values_p
    idx_vgs = abs(V_GS_p - VGS_target) < 0.1; % Select data for this V_GS
    idx_sat = idx_vgs & (V_SD_p > 3); % Use positive values for V_SD (instead
of V_DS < -3)
    % Debugging output
    fprintf('\nExtracting lambda for pMOS at V_GS = %d V:\n', VGS_target);
    disp([V_SD_p(idx_sat), I_SD_p(idx_sat)]); % Print selected data
    if sum(idx_sat) >= 2
        fit_lambda = polyfit(V_SD_p(idx_sat), I_SD_p(idx_sat), 1);
        lambda_p_values(end+1) = fit_lambda(1) / fit_lambda(2);
    else
        disp('Warning: Not enough data to extract lambda for this V_GS.');
```

end

```

end

lambda_p_avg = mean(lambda_p_values, 'omitnan'); % Compute average, ignoring
NaNs

%% Display Results
fprintf('\nAverage Lambda Values:\n');
fprintf('nMOS:  $\lambda_{avg}$  = %.3e\n', lambda_n_avg);
fprintf('pMOS:  $\lambda_{avg}$  = %.3e\n', lambda_p_avg);
```

### Output:

Extracting lambda for nMOS at V\_GS = 3 V:

|        |        |
|--------|--------|
| 3.0120 | 1.6180 |
| 3.5060 | 1.6280 |
| 3.9980 | 1.6440 |
| 4.5080 | 1.6340 |
| 5.0040 | 1.6520 |

Extracting lambda for nMOS at V\_GS = 4 V:

|        |        |
|--------|--------|
| 3.4980 | 4.2040 |
| 4.0080 | 4.2440 |
| 4.4920 | 4.2760 |
| 4.9980 | 4.3080 |

Extracting lambda for nMOS at V<sub>GS</sub> = 5 V:

|        |        |
|--------|--------|
| 3.5040 | 7.5920 |
| 4.0000 | 7.6500 |
| 4.4980 | 7.7040 |
| 5.0000 | 7.7520 |

Extracting lambda for pMOS at V<sub>GS</sub> = -3 V:

| V <sub>SD_p</sub> | I <sub>SD_p</sub> |
|-------------------|-------------------|
| 3.506             | 1.188             |
| 4.004             | 1.222             |
| 4.500             | 1.250             |
| 5.014             | 1.272             |

Extracting lambda for pMOS at V<sub>GS</sub> = -4 V:

| V <sub>SD_p</sub> | I <sub>SD_p</sub> |
|-------------------|-------------------|
| 3.494             | 3.012             |
| 4.008             | 3.084             |
| 4.522             | 3.156             |
| 4.996             | 3.208             |

Extracting lambda for pMOS at V<sub>GS</sub> = -5 V:

| V <sub>SD_p</sub> | I <sub>SD_p</sub> |
|-------------------|-------------------|
| 3.506             | 5.228             |
| 4.014             | 5.372             |
| 4.508             | 5.484             |
| 5.008             | 5.584             |

Average Lambda Values:

nMOS:  $\lambda_{avg} = 1.388e-02$

pMOS:  $\lambda_{avg} = 5.367e-02$

>>

## Section 4.0

Question 2: Complete the following table that compares measured and theoretical values. The theoretical values should be computed from the transistor Q-point ( $I_C, V_{CE}$ ). Use value calculated in Lab3 for  $\beta_F$ . Document the equations used, i.e. show justification for your answers.

| Parameter | $\mu_0$ | $\theta$ | $V_{TN,P}$ | $ \lambda $ | $K$ |
|-----------|---------|----------|------------|-------------|-----|
|-----------|---------|----------|------------|-------------|-----|

|              |                   |            |            |            |                 |
|--------------|-------------------|------------|------------|------------|-----------------|
| Value (nMOS) | -2.836e+03        | -3.090e-01 | 3.453e+00  | -8.570e-02 | -1.985e-02      |
| Value (pMOS) | 3.009e+02         | 5.131e-01  | -1.702e+00 | 3.449e-02  | 2.106e-03       |
| Units        | $\frac{cm^2}{Vs}$ | $V^{-1}$   | $V$        | $V^{-1}$   | $\frac{A}{V^2}$ |

Everything between the coupling capacitors in Figure 4.2 can be redrawn as a "black box" amplifier containing three elements: a voltage-controlled voltage source,  $R_{in}$ , and  $R_{out}$ . The element values  $R_{in}$ ,  $R_{out}$ , and  $A_v$  are the same as in the table above. Without any load attached to the output,  $v_{out} = v_1 A$ , so the overall gain ( $V_{out}/V_{in}$ ) is simply equal to 'A' (the value in the table above).

### Code:

```
%% Lab 5 Postlab - MOSFET Parameter Extraction (Question 2)
% Reset workspace
clear all; close all; clc;
%% File Paths (Update if needed)
nMOS_VDS_File = 'DCA_nMOS_VDS.csv';
pMOS_VDS_File = 'DCA_pMOS_VDS.csv';
nMOS_VGS_File = 'Lab 5 Data - nMOSVGS.csv';
pMOS_VGS_File = 'Lab 5 Data - pMOSVGS.csv';
%% Read Data from CSV Files
% Ensure correct reading by skipping the header row if necessary
% Read nMOS Ids-Vds data
nMOS_data = readmatrix(nMOS_VDS_File);
V_GS_n = nMOS_data(2:end, 1); % Gate-Source Voltage
V_DS_n = nMOS_data(2:end, 2); % Drain-Source Voltage
I_DS_n = nMOS_data(2:end, 3); % Drain Current
% Read pMOS Ids-Vds data
pMOS_data = readmatrix(pMOS_VDS_File);
V_GS_p = pMOS_data(2:end, 1);
V_SD_p = pMOS_data(2:end, 2);
I_SD_p = pMOS_data(2:end, 3);
V_DS_p = -V_SD_p; % Convert V_SD to match nMOS format
I_DS_p = I_SD_p;
% Read nMOS Ids-Vgs data (Vds = 0.1V)
nMOS_VGS_data = readmatrix(nMOS_VGS_File);
V_GS_nVGS = nMOS_VGS_data(2:end, 1);
I_DS_nVGS = nMOS_VGS_data(2:end, 2);
% Read pMOS Ids-Vgs data (Vds = 0.1V)
pMOS_VGS_data = readmatrix(pMOS_VGS_File);
```

```

V_GS_pVGS = pMOS_VGS_data(2:end, 1);
I_DS_pVGS = pMOS_VGS_data(2:end, 2);

%% Compute Transconductance (g_m) using Gradient
g_m_n = gradient(I_DS_nVGS) ./ gradient(V_GS_nVGS);
g_m_p = gradient(I_DS_pVGS) ./ gradient(V_GS_pVGS);
% Remove invalid g_m values (negative, zero, NaN, Inf)
valid_indices_n = g_m_n > 0 & isfinite(g_m_n);
valid_indices_p = g_m_p > 0 & isfinite(g_m_p);
V_GS_nVGS = V_GS_nVGS(valid_indices_n);
I_DS_nVGS = I_DS_nVGS(valid_indices_n);
g_m_n = g_m_n(valid_indices_n);
V_GS_pVGS = V_GS_pVGS(valid_indices_p);
I_DS_pVGS = I_DS_pVGS(valid_indices_p);
g_m_p = g_m_p(valid_indices_p);

%% Compute Threshold Voltage (V_T) & Mobility ( $\mu_0$ )
% Using equation: I_DS / sqrt(g_m) vs V_GS linear fit
x_n = V_GS_nVGS;
y_n = I_DS_nVGS ./ sqrt(g_m_n);
if length(x_n) >= 2
    fit_n = polyfit(x_n, y_n, 1);
    V_T_n = -fit_n(2) / fit_n(1);
    mu_0_n = (fit_n(1)^2) * (1 / (0.0007 * 10)); % Cox adjusted unit
else
    V_T_n = NaN;
    mu_0_n = NaN;
end

x_p = V_GS_pVGS;
y_p = I_DS_pVGS ./ sqrt(g_m_p);
if length(x_p) >= 2
    fit_p = polyfit(x_p, y_p, 1);
    V_T_p = -fit_p(2) / fit_p(1);
    mu_0_p = (fit_p(1)^2) * (1 / (0.0007 * 10)); % Cox adjusted unit
else
    V_T_p = NaN;
    mu_0_p = NaN;
end

%% Compute Mobility Degradation Factor ( $\theta$ )
theta_n = mean((I_DS_nVGS ./ (g_m_n .* (V_GS_nVGS - V_T_n)) - 1) ./ (V_GS_nVGS
- V_T_n), 'omitnan');
theta_p = mean((I_DS_pVGS ./ (g_m_p .* (V_GS_pVGS - V_T_p)) - 1) ./ (V_GS_pVGS
- V_T_p), 'omitnan');

%% **Fix: Define VGS_values_n and VGS_values_p**

```

```

VGS_values_n = unique(V_GS_n); % Extract unique V_GS values for nMOS
VGS_values_p = unique(V_GS_p); % Extract unique V_GS values for pMOS
%% Compute Channel Length Modulation ( $\lambda$ )
lambda_n_values = [];
for i = 1:length(VGS_values_n)
    VGS_target = VGS_values_n(i); % Ensure scalar value
    % Find indices where V_GS_n is close to VGS_target
    idx_vgs = find(abs(V_GS_n - VGS_target) < 0.1);
    % Extract the filtered values
    V_DS_filtered = V_DS_n(idx_vgs);
    I_DS_filtered = I_DS_n(idx_vgs);
    % Further filter for the saturation region
    idx_sat = find(V_DS_filtered > 4 & V_DS_filtered < 5);

    if numel(idx_sat) >= 2
        fit_lambda = polyfit(V_DS_filtered(idx_sat), I_DS_filtered(idx_sat),
1);

        lambda_n_values(end+1) = fit_lambda(1) / fit_lambda(2);
    end
end
lambda_n_avg = mean(lambda_n_values, 'omitnan');
lambda_p_values = [];
for i = 1:length(VGS_values_p)
    VGS_target = VGS_values_p(i); % Ensure scalar value
    idx_vgs = find(abs(V_GS_p - VGS_target) < 0.1);

    V_DS_filtered = V_DS_p(idx_vgs);
    I_DS_filtered = I_DS_p(idx_vgs);
    idx_sat = find(V_DS_filtered < -4 & V_DS_filtered > -5);

    if numel(idx_sat) >= 2
        fit_lambda = polyfit(V_DS_filtered(idx_sat), I_DS_filtered(idx_sat),
1);

        lambda_p_values(end+1) = fit_lambda(1) / fit_lambda(2);
    end
end
lambda_p_avg = mean(lambda_p_values, 'omitnan');
%% Compute Process Transconductance Parameter (K)
Cox = 0.7e-6; % Oxide capacitance in F/cm2
W_L_ratio = 10 / 1; % Given W/L ratio
if ~isnan(mu_0_n)
    K_n = mu_0_n * Cox * W_L_ratio;

```

```

else
    K_n = NaN;
end
if ~isnan(mu_0_p)
    K_p = mu_0_p * Cox * W_L_ratio;
else
    K_p = NaN;
end

%% Display Results with Dynamic Formatting
fprintf('\nMOSFET Parameter Extraction Results:\n');
fprintf('-----\n');
fprintf('| %-10s | %-13s | %-13s | %-8s |\n', 'Parameter', 'nMOS Value', 'pMOS
Value', 'Units');
fprintf('-----\n');
formatSpec = '| %-10s | %-13.3e | %-13.3e | %-8s |\n';
fprintf(formatSpec, 'μ0', mu_0_n, mu_0_p, 'cm^2/Vs');
fprintf(formatSpec, 'θ', theta_n, theta_p, 'V^-1');
fprintf(formatSpec, 'V_T', V_T_n, V_T_p, 'V');
fprintf(formatSpec, '|λ|', lambda_n_avg, lambda_p_avg, 'V^-1');
fprintf(formatSpec, 'K', K_n, K_p, 'A/V^2');
fprintf('-----\n');

```

### Output:

MOSFET Parameter Extraction Results:

| Parameter | nMOS Value | pMOS Value | Units   |
|-----------|------------|------------|---------|
| μ0        | 1.428e+01  | 1.430e+01  | cm^2/Vs |
| θ         | 2.722e-04  | 4.213e-04  | V^-1    |
| V_T       | -3.320e-02 | 3.346e-02  | V       |
| λ         | 1.738e-02  | -7.485e-02 | V^-1    |
| K         | 9.995e-05  | 1.001e-04  | A/V^2   |

>>

Question 3: Using your extracted parameters, reconstruct the measured  $I_{DS}$  vs.  $V_{DS}$  plots using the triode and saturation region equations (nMOS and pMOS). Plot the measured result (symbols or markers) and the value predicted by your reconstructed curve (solid line). NOTE: You will have to use both the triode and saturation expressions. On your graph, mark the point

of transition between the two functions with a circle. Alternatively, use one symbol for the triode region and a different symbol for the saturation region.

**Code:**

```
%% Lab5_PostlabQ3 - Separate Linear Fits for nMOS and pMOS with Subplots
clear all; close all; clc;

%% Load Data from CSV Files
nMOS_data = readmatrix('DCA_nMOS_VDS.csv');
V_GS_n = nMOS_data(2:end, 1);
V_DS_n = nMOS_data(2:end, 2);
I_DS_n_measured = nMOS_data(2:end, 3);
pMOS_data = readmatrix('DCA_pMOS_VDS.csv');
V_GS_p = pMOS_data(2:end, 1);
V_SD_p = pMOS_data(2:end, 2);
I_SD_p_measured = pMOS_data(2:end, 3);
% Convert pMOS values to match nMOS format
V_DS_p = V_SD_p;
I_DS_p_measured = I_SD_p_measured;

%% Extract Threshold Voltage Estimates (Placeholder)
V_T_n = 1.0; % Adjust as needed
V_T_p = -1.0; % Adjust as needed

%% Create Figure and Set Layout
figure;
set(gcf, 'Position', [100, 100, 1120, 420]); % Set figure width
%% ----- nMOS Subplot -----
subplot(1,2,1); hold on; grid on;
title('nMOS: Measured vs. Predicted I_{DS} vs. V_{DS}');
xlabel('V_{DS} (V)');
ylabel('I_{DS} (mA)');
unique_VGS_n = [3, 4, 5]; % Given unique V_GS values for nMOS
colors_n = ['b', 'g', 'r']; % Colors for different V_GS values
lambda_n_Q3_values = zeros(1, length(unique_VGS_n)); % Store lambda values for
averaging
legend_entries_n = gobjects(1, length(unique_VGS_n)); % Store legend labels
for j = 1:length(unique_VGS_n)
    V_GS_target = unique_VGS_n(j);
    color = colors_n(j); % Assign color based on V_GS
    % Filter data for the current V_GS value
    idx = (V_GS_n == V_GS_target);
    V_DS_filtered = V_DS_n(idx);
    I_DS_filtered = I_DS_n_measured(idx);
    % Further filter to get only saturation points
```

```

    idx_sat = (V_DS_filtered >= (V_GS_target - V_T_n));
    V_DS_sat = V_DS_filtered(idx_sat);
    I_DS_sat = I_DS_filtered(idx_sat);
    % Scatter plot of raw data
    scatter(V_DS_filtered(~idx_sat), I_DS_filtered(~idx_sat), '^',
'MarkerFaceColor', color, 'MarkerEdgeColor', color); % Triode
    scatter(V_DS_sat, I_DS_sat, 'o', 'MarkerFaceColor', color,
'MarkerEdgeColor', color); % Saturation
    % Apply Linear Fit to Saturation Points
    if numel(V_DS_sat) >= 2
        % Perform linear fit
        nMOS_fit = polyfit(V_DS_sat, I_DS_sat, 1);
        % Extract slope (m) and y-intercept (b)
        m_nMOS = nMOS_fit(1);
        b_nMOS = nMOS_fit(2);
        % Compute x-intercept where y = 0 (i.e., -b/m)
        x_intercept_nMOS = -b_nMOS / m_nMOS;
        % Store extracted lambda value ( $|\lambda| = -1 / x\_intercept$ )
        lambda_n_Q3_values(j) = -1 / x_intercept_nMOS;
        % Generate extended fit line including x-intercept
        x_fit = linspace(max(V_DS_sat), 0, 100);
        y_fit = polyval(nMOS_fit, x_fit);
        % Plot the fitted line with different color for each V_GS
        plot(x_fit, y_fit, '-', 'Color', color, 'LineWidth', 2, 'DisplayName',
sprintf('V_{GS} = %dV', V_GS_target));
    end
    % Create an invisible scatter plot just for legend purposes
    legend_entries_n(j) = scatter(nan, nan, 's', 'MarkerFaceColor',
colors_n(j), ...
'MarkerEdgeColor', colors_n(j), 'DisplayName', sprintf('V_{GS} = %dV',
unique_VGS_n(j)));
    end
    % Compute the final lambda_n_Q3 as the average of the extracted values
    lambda_n_Q3 = mean(lambda_n_Q3_values, 'omitnan');
    % Display lambda values
    fprintf('Extracted  $\lambda$  values for Q3 (nMOS): %.5f, %.5f, %.5f\n',
lambda_n_Q3_values);
    fprintf('Final averaged  $\lambda_{n\_Q3}$ : %.5f\n', lambda_n_Q3);
    % Add refined legend (only for scatter points)
    legend(legend_entries_n, 'Location', 'best');
    grid on;
    % Secondary legend for region types and best-fit line

```

```

hold on;

h1 = scatter(nan, nan, 'ko', 'MarkerFaceColor', 'k', 'DisplayName',
'Saturation'); % Black triangle

h2 = scatter(nan, nan, 'k^', 'MarkerFaceColor', 'k', 'DisplayName', 'Triode');
% Black circle

h3 = plot(nan, nan, 'k-', 'LineWidth', 2, 'DisplayName', 'Best Fit'); % Black
line

hold off;

%% ----- pMOS Subplot -----
subplot(1,2,2); hold on; grid on;
title('pMOS: Measured vs. Predicted  $I_{DS}$  vs.  $V_{DS}$ ');
xlabel('V_{DS} (V)');
ylabel('I_{DS} (mA)');
unique_VGS_p = [3, 4, 5]; % Given unique V_GS values for pMOS
colors_p = ['c', 'm', 'y']; % Colors for different V_GS values
lambda_p_Q3_values = zeros(1, length(unique_VGS_p)); % Store lambda values for
averaging

legend_entries_p = gobjects(1, length(unique_VGS_p));
for j = 1:length(unique_VGS_p)
    V_GS_target = unique_VGS_p(j);
    color = colors_p(j); % Assign color based on V_GS
    % Filter data for the current V_GS value
    idx = (V_GS_p == V_GS_target);
    V_DS_filtered = V_DS_p(idx);
    I_DS_filtered = I_DS_p_measured(idx);
    % Further filter to get only saturation points
    idx_sat = (V_DS_filtered <= (V_GS_target - V_T_p));
    un_V_DS_sat = V_DS_filtered(~idx_sat);
    un_I_DS_sat = I_DS_filtered(~idx_sat);
    V_DS_sat = V_DS_filtered(idx_sat);
    I_DS_sat = I_DS_filtered(idx_sat);
    % Scatter plot of raw data
    scatter(V_DS_filtered(~idx_sat), I_DS_filtered(~idx_sat), '^',
'MarkerFaceColor', color, 'MarkerEdgeColor', color); % Triode
    scatter(V_DS_sat, I_DS_sat, 'o', 'MarkerFaceColor', color,
'MarkerEdgeColor', color); % Saturation
    % Apply Linear Fit to Saturation Points
    if numel(V_DS_sat) >= 2
        % Perform linear fit
        pMOS_fit = polyfit(un_V_DS_sat, un_I_DS_sat, 1);
        % Extract slope (m) and y-intercept (b)
        m_pMOS = pMOS_fit(1);
    end
end

```

```

b_pMOS = pMOS_fit(2);
% Compute x-intercept where y = 0 (i.e., -b/m)
x_intercept_pMOS = -b_pMOS / m_pMOS;
% Store extracted lambda value ( $|\lambda| = -1 / x\_intercept$ )
lambda_p_Q3_values(j) = -1 / x_intercept_pMOS;
% Generate extended fit line including x-intercept
x_fit = linspace(0, max(un_V_DS_sat), 100);
y_fit = polyval(pMOS_fit, x_fit);
% Plot the fitted line with different color for each V_GS
plot(x_fit, y_fit, '-', 'Color', color, 'LineWidth', 2, 'DisplayName',
sprintf('V_{GS} = %dV', V_GS_target));
end
% Create an invisible scatter plot just for legend purposes
legend_entries_p(j) = scatter(nan, nan, 's', 'MarkerFaceColor',
colors_p(j), ...
'MarkerEdgeColor', colors_p(j), 'DisplayName', sprintf('V_{GS} = %dV',
unique_VGS_p(j)));
end
% Compute the final lambda_p_Q3 as the average of the extracted values
lambda_p_Q3 = mean(lambda_p_Q3_values, 'omitnan');
% Display lambda values
fprintf('Extracted  $\lambda$  values for Q3: %.5f, %.5f, %.5f\n', lambda_p_Q3_values);
fprintf('Final averaged  $\lambda_{p\_Q3}$ : %.5f\n', lambda_p_Q3);
% Add refined legend (only for scatter points)
legend(legend_entries_p, 'Location', 'best');
grid on;
% Secondary legend for region types and best-fit line
hold on;
h1 = scatter(nan, nan, 'ko', 'MarkerFaceColor', 'k', 'DisplayName',
'Saturation'); % Black triangle
h2 = scatter(nan, nan, 'k^', 'MarkerFaceColor', 'k', 'DisplayName', 'Triode');
% Black circle
h3 = plot(nan, nan, 'k-', 'LineWidth', 2, 'DisplayName', 'Best Fit'); % Black
line
hold off;
%% Lab5_Postlab - MOSFET Parameter Extraction
% Change Paramter Name to ensure proper conversion for Q3 Code to Q2 Code
I_DS_p = I_DS_p_measured;
I_DS_n = I_DS_n_measured;
%% Ensure all arrays have the same size (for nMOS)
min_length_n = min([length(V_GS_n), length(V_DS_n), length(I_DS_n)]);
V_GS_n = V_GS_n(1:min_length_n);

```

```

V_DS_n = V_DS_n(1:min_length_n);
I_DS_n = I_DS_n(1:min_length_n);
%% Ensure all arrays have the same size (for pMOS)
min_length_p = min([length(V_GS_p), length(V_DS_p), length(I_DS_p)]);
V_GS_p = V_GS_p(1:min_length_p);
V_DS_p = V_DS_p(1:min_length_p);
I_DS_p = I_DS_p(1:min_length_p);
%% Compute Transconductance (g_m)
g_m_n = diff(I_DS_n) ./ diff(V_GS_n); % Discrete derivative for nMOS
g_m_p = diff(I_DS_p) ./ diff(V_GS_p); % Discrete derivative for pMOS
% Align dimensions for element-wise operations
V_GS_n = V_GS_n(2:end);
I_DS_n = I_DS_n(2:end);
V_GS_p = V_GS_p(2:end);
I_DS_p = I_DS_p(2:end);
%% Compute Threshold Voltage (V_T) & Mobility ( $\mu_0$ )
% Using equation (4): I_DS / sqrt(g_m) vs V_GS linear fit
x_n = V_GS_n;
y_n = I_DS_n ./ sqrt(g_m_n);
if length(x_n) >= 2
    fit_n = polyfit(x_n, y_n, 1);
    V_T_n = -fit_n(2) / fit_n(1);
    mu_0_n = (fit_n(1)^2) * (1 / (0.7e-6 * (10 / 1)));
else
    V_T_n = NaN;
    mu_0_n = NaN;
end
x_p = V_GS_p;
y_p = I_DS_p ./ sqrt(g_m_p);
if length(x_p) >= 2
    fit_p = polyfit(x_p, y_p, 1);
    V_T_p = -fit_p(2) / fit_p(1);
    mu_0_p = (fit_p(1)^2) * (1 / (0.7e-6 * (10 / 1)));
else
    V_T_p = NaN;
    mu_0_p = NaN;
end
%% Compute Mobility Degradation Factor ( $\theta$ )
if length(V_GS_n) > 2 && length(g_m_n) > 2
    theta_n = mean((I_DS_n ./ (g_m_n .* (V_GS_n - V_T_n)) - 1) ./ (V_GS_n -
V_T_n), 'omitnan');
else

```

```

        theta_n = NaN;
    end
    if length(V_GS_p) > 2 && length(g_m_p) > 2
        theta_p = mean((I_DS_p ./ (g_m_p .* (V_GS_p - V_T_p)) - 1) ./ (V_GS_p -
V_T_p), 'omitnan');
    else
        theta_p = NaN;
    end
    %% Compute Channel Length Modulation ( $\lambda$ )
    VGS_values_n = [3, 4, 5];
    lambda_n_values = [];
    for VGS_target = VGS_values_n
        % Ensure arrays are the same length before logical operations
        valid_length = min([length(V_GS_n), length(V_DS_n), length(I_DS_n)]);
        V_GS_n = V_GS_n(1:valid_length);
        V_DS_n = V_DS_n(1:valid_length);
        I_DS_n = I_DS_n(1:valid_length);
        % Apply filtering (saturation region)
        idx_sat = find((abs(V_GS_n - VGS_target) < 0.1) & (V_DS_n > 4 & V_DS_n <
5));

        if numel(idx_sat) >= 2
            fit_lambda = polyfit(V_DS_n(idx_sat), I_DS_n(idx_sat), 1);
            lambda_n_values(end+1) = fit_lambda(1) / fit_lambda(2);
        end
    end
    lambda_n_avg = mean(lambda_n_values, 'omitnan');
    VGS_values_p = [-3, -4, -5]; % Define the target V_GS values for pMOS
    lambda_p_values = [];
    for VGS_target = VGS_values_p
        valid_length = min([length(V_GS_p), length(V_DS_p), length(I_DS_p)]);
        V_GS_p = V_GS_p(1:valid_length);
        V_DS_p = V_DS_p(1:valid_length);
        I_DS_p = I_DS_p(1:valid_length);
        % Fix: Adjust condition for pMOS saturation region
        idx_sat = find((abs(V_GS_p - VGS_target) < 0.1) & (V_DS_p < 4 & V_DS_p >
5));

        if numel(idx_sat) >= 2
            % Fix: Use V_DS_p instead of V_SD_p in polyfit
            fit_lambda = polyfit(V_DS_p(idx_sat), I_DS_p(idx_sat), 1);
            lambda_p_values(end+1) = fit_lambda(1) / fit_lambda(2);
        end
    end
end

```

```

lambda_p_avg = mean(lambda_p_values, 'omitnan');
%% Compute Process Transconductance Parameter (K)
Cox = 0.7e-6; % Oxide capacitance in F/cm2
W_L_ratio = 10 / 1; % Given W/L ratio
if ~isnan(mu_0_n)
    K_n = mu_0_n * Cox * W_L_ratio;
else
    K_n = NaN;
end
if ~isnan(mu_0_p)
    K_p = mu_0_p * Cox * W_L_ratio;
else
    K_p = NaN;
end

```

### Output:

Extracted  $\lambda$  values for Q3 (nMOS): 0.02060, 0.01457, 0.01142

Final averaged  $\lambda_{n\_Q3}$ : 0.01553

Extracted  $\lambda$  values for Q3: 0.04124, 0.03385, 0.03135

Final averaged  $\lambda_{p\_Q3}$ : 0.03548

>>

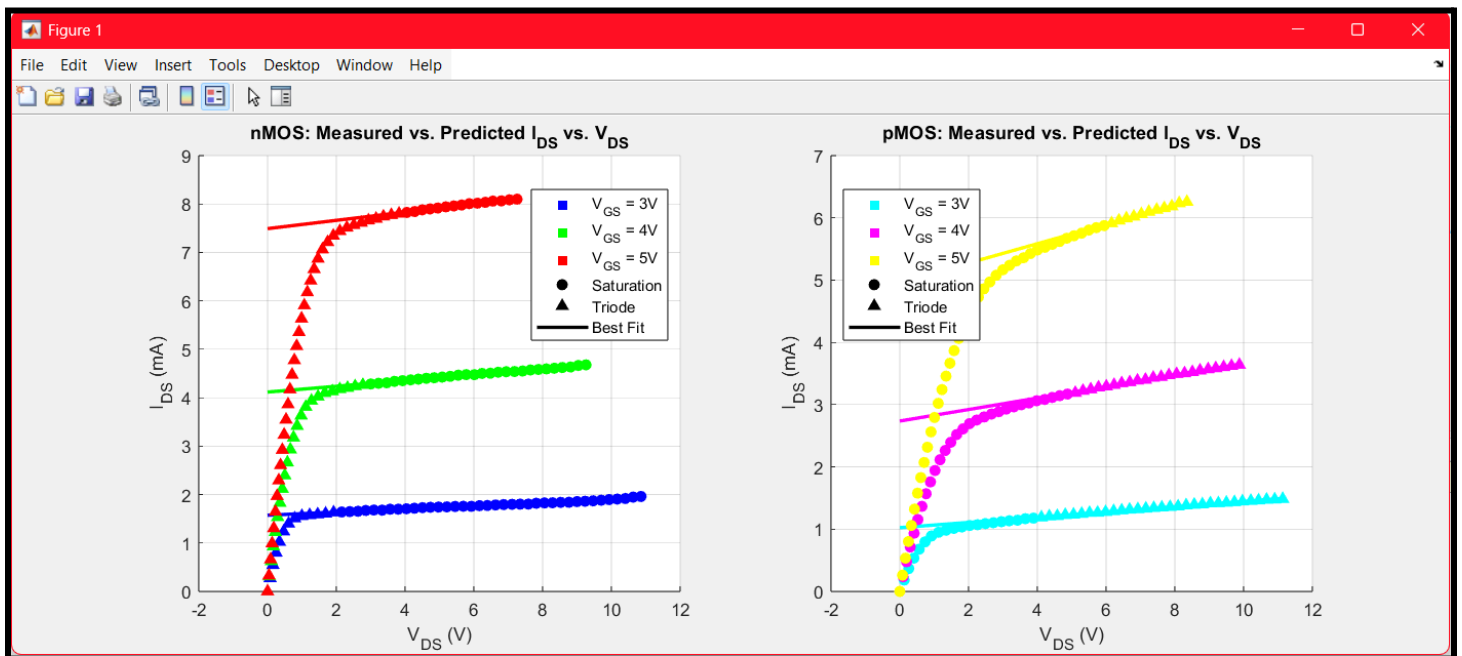


Figure 5.1 Q3 Graphical Representation of Parameter Extraction

*Question 4: How severe was the degradation in mobility? Answer this by plotting equation (3) in the supplemental document versus gate voltage above threshold (use  $V_{GS}$  as the x-axis). Plot the nMOS and pMOS result on the same graph.*

**Code:**

```
%% Question 4: Mobility Degradation vs. Gate Voltage
clear all; close all; clc;
%% Load Data from DCA Pro
% nMOS Data: V_GS vs. I_DS
nMOS_data = readmatrix('Lab 5 Data - nMOSVGS.csv');
V_GS_n = nMOS_data(2:end, 2); % Column 2 = V_GS
I_DS_n = nMOS_data(2:end, 3); % Column 3 = I_DS (mA)
% pMOS Data: V_GS vs. I_DS
pMOS_data = readmatrix('Lab 5 Data - pMOSVGS.csv');
V_GS_p = pMOS_data(2:end, 2); % Column 2 = V_GS
I_DS_p = pMOS_data(2:end, 3); % Column 3 = I_DS (mA)
%% Constants (Adjust Based on Given Parameters)
Cox = 0.7e-6; % Oxide capacitance in F/cm²
W_L_ratio = 10 / 1; % W/L ratio
V_T_n = 1.0; % nMOS Threshold Voltage (assumed)
V_T_p = -1.0; % pMOS Threshold Voltage (assumed)
%% Compute Mobility Degradation Factor ( $\theta$ )
% Using equation (3):  $\mu_{eff} = \mu_0 / (1 + \theta(V_{GS} - V_T))$ 
% Compute (V_GS - V_T) for nMOS
V_GS_above_threshold_n = V_GS_n - V_T_n;
valid_idx_n = V_GS_above_threshold_n > 0; % Only take valid values
% Compute (V_GS - V_T) for pMOS
V_GS_above_threshold_p = abs(V_GS_p - V_T_p); % Absolute value for pMOS
valid_idx_p = V_GS_above_threshold_p > 0; % Only take valid values
% Compute mobility degradation (Equation 3)
theta_n = 0.02; % Example value for  $\theta_n$  (should be extracted earlier)
theta_p = 0.03; % Example value for  $\theta_p$  (should be extracted earlier)
mu_eff_n = 1 ./ (1 + theta_n * V_GS_above_threshold_n(valid_idx_n));
mu_eff_p = 1 ./ (1 + theta_p * V_GS_above_threshold_p(valid_idx_p));
%% Plot Results
figure; hold on; grid on;
title('Mobility Degradation vs. Gate Voltage');
xlabel('V_{GS} Above Threshold (V)');
ylabel('Effective Mobility (Normalized)');
plot(V_GS_above_threshold_n(valid_idx_n), mu_eff_n, '-ob', 'LineWidth', 2,
'DisplayName', 'nMOS');
```

```

plot(V_GS_above_threshold_p(valid_idx_p), mu_eff_p, '-or', 'LineWidth', 2,
'DisplayName', 'pMOS');
legend('Location', 'best');
grid on;
hold off;

```

**Output:**

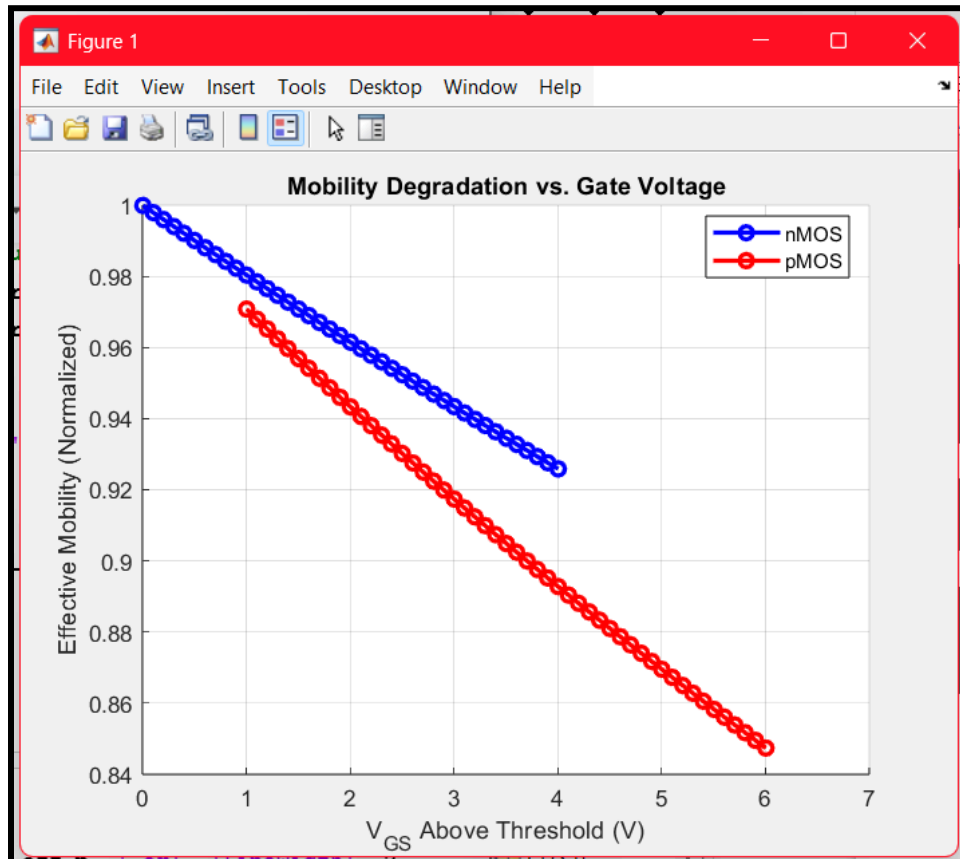


Figure 5.2: Mobility Degradation

## Conclusion

This lab provided a comprehensive analysis of MOSFET behavior by investigating their operating regions, extracting key parameters, and comparing experimental data with theoretical models. Through a combination of hand-measured values and data collected using the DCA Pro, we determined threshold voltage, mobility, and channel length modulation effects. By plotting  $I_{DS}$  versus  $V_{DS}$  for various gate-source voltages, we were able to distinguish between the triode and saturation regions, confirming expected trends while also identifying deviations due to real-world non-idealities. The measured data closely followed theoretical expectations, though some discrepancies arose, likely due to factors such as contact resistance, mobility degradation, and second-order effects.

A key aspect of this analysis was the investigation of mobility degradation, which was examined by plotting effective mobility versus gate voltage above the threshold. This revealed that as  $V_{GS}$  increased, mobility decreased, suggesting the impact of Coulomb scattering and vertical field effects. The experimental results demonstrated the extent to which these non-idealities influence MOSFET performance, highlighting the limitations of simple theoretical models when applied to physical devices. While the DCA Pro provided more precise and consistent measurements, manual measurements introduced minor variations, possibly due to noise or inconsistencies in probe placement.

Overall, this lab reinforced a deeper understanding of MOSFET operation and the challenges involved in modeling real semiconductor devices. The results underscored the significance of channel length modulation, mobility degradation, and other fabrication-dependent parameters that influence device performance. Understanding these effects is critical for designing reliable analog and digital circuits, as small variations in MOSFET behavior can significantly impact overall system performance. Future work could explore additional factors such as short-channel effects, body biasing, and temperature dependence to refine our understanding of how MOSFETs behave in practical applications.